

ログ基盤 - 詳細説明書

このドキュメントについて

このドキュメントは、Milestone 2の基盤2「ログ基盤」の詳細な技術説明書です。全操作の自動記録、コード位置の自動検出、システム復旧機能について包括的に説明しています。

対象読者: 開発者、技術リード、システム管理者

難易度: 中級～上級

推定読了時間: 20-25分

概要

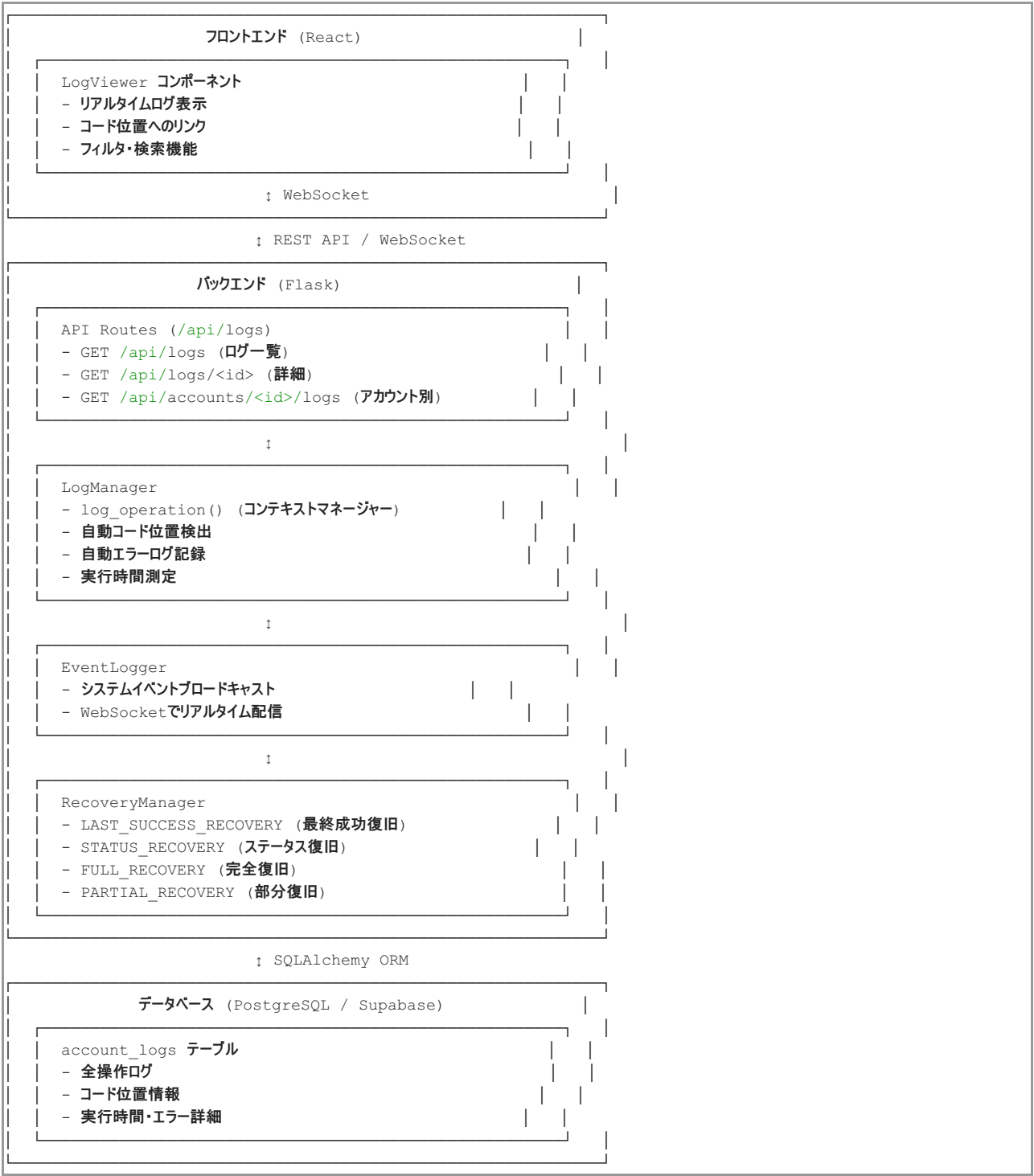
Milestone 2の第2の基盤として、200アカウントの全操作を記録・追跡し、システム障害時の完全復旧を可能にするログ基盤を構築しました。このシステムは、自動コード位置検出、コンテキストマネージャーパターン、4つの復旧戦略を提供します。

技術スタック

- バックエンド: Python + Flask
- ログ記録: Python `inspect` モジュール
- データベース: Supabase (PostgreSQL Online) + JSONB
- リアルタイム配信: Flask-SocketIO + WebSocket
- フロントエンド: TypeScript + React + Socket.IO Client

システムアーキテクチャ

全体構成図



コア機能

1. 自動コード位置検出

実装原理

```
def _get_code_location() -> Dict[str, Optional[Any]]:
    """
    現在のコード位置を自動検出

    呼び出しスタック:
    Frame 0: _get_code_location() (現在)
    Frame 1: LogContext.log_success() (LogManager内部)
    Frame 2: ユーザーコード (実際のログ呼び出し元) ← 取得したい
    """
    frame = inspect.currentframe()
    if frame:
        # 2つ前のフレームを取得(ユーザーコード)
        caller_frame = frame.f_back.f_back
        if caller_frame:
            file_path = caller_frame.f_code.co_filename

            # プロジェクトルートからの相対パスに変換
            project_root = Path(__file__).parent.parent.parent
            relative_path = os.path.relpath(file_path, project_root)

            return {
                'file_path': relative_path.replace('\\', '/'),
                'function_name': caller_frame.f_code.co_name,
                'line_number': caller_frame.f_lineno
            }
    
```

スタックフレーム構造



データベース保存構造

```
CREATE TABLE account_logs (  
  id UUID PRIMARY KEY,  
  account_id UUID NOT NULL,  
  action_type VARCHAR(50) NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  message TEXT,  
  
  -- コード位置情報(自動記録)  
  file_path VARCHAR(500),      -- 'src/automation/login_handler.py'  
  function_name VARCHAR(200),  -- 'login'  
  line_number INTEGER,         -- 85  
  
  -- 実行情報  
  execution_time FLOAT,        -- 0.523 (秒)  
  created_at TIMESTAMP NOT NULL,  
  
  -- エラー詳細(JSONB)  
  error_detail JSONB  
);
```

2. コンテキストマネージャーパターン

実装構造

```

class LogManagerer:
    @contextmanager
    def log_operation(self, account_id, action_type, **extra_data):
        """操作をログ記録するコンテキストマネージャー"""
        start_time = time.time()
        request_id = str(uuid.uuid4())

        # ログコンテキストを作成
        log_context = LogContext(
            account_id=account_id,
            action_type=action_type,
            request_id=request_id,
            start_time=start_time
        )

        try:
            # 操作開始イベント
            event_logger.log_event(
                event_type=EventType.LOGIN_STARTED,
                severity=EventSeverity.INFO,
                account_id=account_id,
                message=f"{action_type.value} operation started"
            )

            yield log_context # ユーザーコードが実行される

        except Exception as e:
            # 自動エラーログ記録
            execution_time = time.time() - start_time
            log_context.log_error(
                message=str(e),
                error_detail={
                    'exception_type': type(e).__name__,
                    'traceback': str(e)
                },
                execution_time=execution_time
            )
            raise

        finally:
            # 未完了の場合はスキップログを記録
            if not log_context.completed:
                execution_time = time.time() - start_time
                log_context.log_skipped(
                    message="Operation was not completed",
                    execution_time=execution_time
                )

```

使用例と実行フロー

```

# ユーザーコード
with log_manager.log_operation(account_id, ActionType.LOGIN) as log_ctx:
    # ログイン処理
    page.fill("input[name='username']", username)
    page.fill("input[type='password']", password)
    page.click("button[type='submit']")

    # 成功をログ記録
    log_ctx.log_success("Login successful")

```

実行フローの詳細

1. コンテキストマネージャー開始

```
start_time = time.time() # 開始時刻を記録
request_id = uuid4()      # リクエストIDを生成
```

↓

2. 操作開始イベント発行

```
event_logger.log_event(
    event_type=LOGIN_STARTED,
    severity=INFO,
    message="LOGIN operation started"
)
```

↓

3. ユーザーコード実行

```
page.fill("input[name='username']", username)
page.fill("input[type='password']", password)
page.click("button[type='submit']")
```

↓

4. 成功ログ記録

```
log_ctx.log_success("Login successful")
```

↓

コード位置を自動検出

```
- file_path: 'src/automation/login_handler.py'
- function_name: 'login'
- line_number: 85
```

↓

データベースに保存

```
INSERT INTO account_logs (...) VALUES (...)
```

↓

5. コンテキストマネージャー終了

```
execution_time = time.time() - start_time # 実行時間
# 自動的にクリーンアップ
```

エラー時の自動ログ記録

```
try:
    with log_manager.log_operation(account_id, ActionType.LOGIN) as log_ctx:
        # エラーが発生
        raise Exception("Login failed: Invalid credentials")
except Exception as e:
    # コンテキストマネージャーが自動的にエラーをログ記録
    # ユーザーは何もする必要がない
    pass
```

自動記録される内容:

```
{
  "account_id": "550e8400-e29b-41d4-a716-446655440000",
  "action_type": "LOGIN",
  "status": "FAILURE",
  "message": "Login failed: Invalid credentials",
  "error_detail": {
    "exception_type": "Exception",
    "traceback": "...",
    "stack_trace": "..."
  },
  "file_path": "src/automation/login_handler.py",
  "function_name": "login",
  "line_number": 85,
  "execution_time": 2.341,
  "created_at": "2025-01-10T08:30:15.123Z"
}
```

3. イベントシステム

イベントタイプ

```
class EventType(str, Enum):
    # アカウント操作
    ACCOUNT_CREATED = 'ACCOUNT_CREATED'
    ACCOUNT_UPDATED = 'ACCOUNT_UPDATED'
    ACCOUNT_STATUS_CHANGED = 'ACCOUNT_STATUS_CHANGED'

    # ログイン操作
    LOGIN_STARTED = 'LOGIN_STARTED'
    LOGIN_SUCCESS = 'LOGIN_SUCCESS'
    LOGIN_FAILED = 'LOGIN_FAILED'

    # スケジュール操作
    SCHEDULE_UPDATE_STARTED = 'SCHEDULE_UPDATE_STARTED'
    SCHEDULE_UPDATE_COMPLETED = 'SCHEDULE_UPDATE_COMPLETED'
    SCHEDULE_UPDATE_FAILED = 'SCHEDULE_UPDATE_FAILED'

    # 待機接客操作
    WAIT_RECEPTION_STARTED = 'WAIT_RECEPTION_STARTED'
    WAIT_RECEPTION_COMPLETED = 'WAIT_RECEPTION_COMPLETED'
    WAIT_RECEPTION_FAILED = 'WAIT_RECEPTION_FAILED'

    # エラー
    ERROR_OCCURRED = 'ERROR_OCCURRED'
    ERROR_RECOVERED = 'ERROR_RECOVERED'

    # セッション
    SESSION_CONFLICT_DETECTED = 'SESSION_CONFLICT_DETECTED'
```

イベント配信フロー

1. イベント発生

```
event_logger.log_event(  
    event_type=LOGIN_SUCCESS,  
    severity=INFO,  
    account_id="...",  
    message="Login successful"  
)
```

↓

2. イベントオブジェクト作成

```
event = SystemEvent(  
    event_id=uuid4(),  
    event_type=LOGIN_SUCCESS,  
    timestamp=datetime.utcnow(),  
    ...  
)
```

↓



イベントの構造

```
@dataclass  
class SystemEvent:  
    event_id: str # イベントID (UUID)  
    event_type: EventType # イベントタイプ  
    severity: EventSeverity # 重要度 (INFO/WARNING/ERROR/CRITICAL)  
    account_id: Optional[str] # アカウントID  
    timestamp: datetime # 発生時刻  
    message: str # メッセージ  
    data: Dict[str, Any] # 追加データ  
    request_id: Optional[str] # リクエストID(トレーシング用)
```

4. 復旧システム

4つの復旧戦略

復旧戦略の選択

1. LAST_SUCCESS_RECOVERY

最後の成功状態に復旧

- └ 最終成功ログを検索
- └ ログイン情報を復元
- └ ステータスを `IDLE` に変更

2. STATUS_RECOVERY

ステータスのみ復旧

- └ エラーカウントをリセット
- └ ステータスを `IDLE` に変更
- └ `is_active` を `TRUE` に変更

3. FULL_RECOVERY

完全復旧(全フィールド)

- └ ログイン情報を復元
- └ セッション情報を復元
- └ ステータスを復元
- └ エラー情報をクリア

4. PARTIAL_RECOVERY

部分復旧(選択的復旧)

- └ ユーザー指定のフィールドのみ
- └ 手動確認が必要な場合

LAST_SUCCESS_RECOVERY の実装

```

def _recover_from_last_success(
    self,
    account: Account,
    account_repo: AccountRepository,
    log_repo: AccountLogRepository
) -> Dict[str, Any]:
    """最後の成功状態から復旧"""

    # 1. 最後の成功ログを検索
    success_logs = log_repo.get_account_logs(
        account_id=account.id,
        limit=100
    )

    # 2. LOGIN SUCCESS を探す
    last_success_login = None
    for log in success_logs:
        if (log.action_type == ActionType.LOGIN.value and
            log.status == LogStatus.SUCCESS.value):
            last_success_login = log
            break

    if last_success_login:
        # 3. ログイン情報を復元
        login_url = last_success_login.error_detail.get('login_url')
        session_token = last_success_login.error_detail.get('session_token')

        # 4. アカウント情報を更新
        account_repo.update(account.id,
            login_url=login_url,
            session_token=session_token,
            status=AccountStatus.IDLE,
            error_count=0,
            last_error=None
        )

    return {
        'recovered_fields': ['login_url', 'session_token', 'status'],
        'errors': []
    }

```

復旧フロー図

エラー発生

```
account.status = ERROR
account.error_count = 5
account.is_active = FALSE
```

↓

復旧開始

```
recovery_manager.recover_account_from_logs(
    account_id,
    strategy=RecoveryStrategy.LAST_SUCCESS_RECOVERY
)
```

↓

ログ分析

最近100件のログを取得

↓

最後の LOGIN SUCCESS ログを検索

Log Entry (2時間前)

```
action_type: LOGIN
status: SUCCESS
error_detail: {
  "login_url": "https://doors1.shinchakun.info/...",
  "session_token": "abc123..."
}
```

↓

状態復元

```
UPDATE accounts SET
    login_url = 'https://doors1.shinchakun.info/...',
    session_token = 'abc123...',
    status = 'IDLE',
    error_count = 0,
    last_error = NULL,
    is_active = TRUE
WHERE id = '...'
```

↓

復旧完了

```
account.status = IDLE
account.error_count = 0
account.is_active = TRUE
```

データベース設計

account_logs テーブル

```
CREATE TABLE account_logs (  
  -- プライマリキー  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  -- 関連  
  account_id UUID NOT NULL REFERENCES accounts(id),  
  
  -- 操作情報  
  action_type VARCHAR(50) NOT NULL, -- LOGIN, SCHEDULE_UPDATE, etc.  
  status VARCHAR(20) NOT NULL,      -- SUCCESS, FAILURE, SKIPPED  
  message TEXT,  
  
  -- エラー詳細(JSONB形式)  
  error_detail JSONB,  
  
  -- 実行情報  
  execution_time FLOAT,              -- 実行時間(秒)  
  request_id VARCHAR(100),          -- リクエストID(トレーシング用)  
  
  -- コード位置情報(自動記録)  
  file_path VARCHAR(500),           -- ファイルパス  
  function_name VARCHAR(200),       -- 関数名  
  line_number INTEGER,              -- 行番号  
  
  -- タイムスタンプ  
  created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),  
  
  -- インデックス  
  INDEX idx_account_logs_account_id (account_id),  
  INDEX idx_account_logs_action_type (action_type),  
  INDEX idx_account_logs_status (status),  
  INDEX idx_account_logs_created_at (created_at),  
  INDEX idx_account_logs_request_id (request_id)  
);
```

テーブル構造の視覚化

account_logs テーブル	
id (UUID)	PRIMARY KEY
account_id (UUID)	FOREIGN KEY, INDEXED
action_type (VARCHAR)	INDEXED
status (VARCHAR)	INDEXED
message (TEXT)	
error_detail (JSONB)	{exception_type, trace...}
execution_time (FLOAT)	秒単位
request_id (VARCHAR)	INDEXED (トレーシング用)
file_path (VARCHAR)	[自動記録]
function_name (VARCHAR)	[自動記録]
line_number (INTEGER)	[自動記録]
created_at (TIMESTAMP)	INDEXED

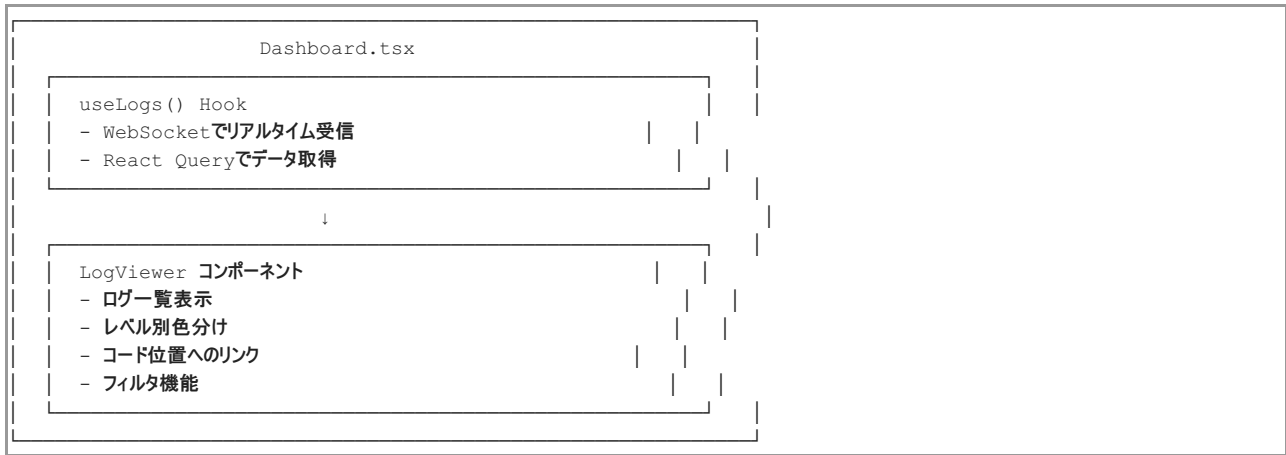
ログエントリの例

```
{
  "id": "a1b2c3d4-...",
  "account_id": "550e8400-...",
  "action_type": "LOGIN",
  "status": "SUCCESS",
  "message": "Login successful",
  "error_detail": {
    "login_url": "https://doors1.shinchakun.info/dokodemo/#/",
    "session_token": "abc123...",
    "user_agent": "Mozilla/5.0..."
  },
  "execution_time": 2.341,
  "request_id": "req-12345",
  "file_path": "src/automation/login_handler.py",
  "function_name": "login",
  "line_number": 85,
  "created_at": "2025-01-10T08:30:15.123Z"
}
```

フロントエンド統合

リアルタイムログ表示

コンポーネント構造



UI表示例

ログビューア					フィルタ: [すべて▼]
時刻	アカウント	レベル	メッセージ	詳細	
08:30:15	user001	成功	Login successful	login_handler.py:85	
08:30:10	user002	警告	Manual operation	manual_detector.py:82	
08:29:58	user003	エラー	Login failed	login_handler.py:120	

コード位置へのリンク機能

```
// LogEntry コンポーネント
function LogEntry({ log }: { log: LogEntry }) {
  const handleClick = async () => {
    // バックエンドからコード内容を取得
    const response = await fetch(
      `/api/code-viewer?file=${log.filePath}&line=${log.lineNumber}`
    );
    const codeContent = await response.json();

    // モーダルでコードを表示
    showCodeViewer({
      filePath: log.filePath,
      functionName: log.functionName,
      lineNumber: log.lineNumber,
      code: codeContent.code,
      highlightLine: log.lineNumber
    });
  };

  return (
    <div className="log-entry">
      <span className={`level-${log.level}`}>
        {log.level === 'success' && '✅'}
        {log.level === 'warning' && '⚠️'}
        {log.level === 'error' && '❌'}
      </span>
      <span>{log.message}</span>
      <button onClick={handleClick} className="code-link">
        📄 {log.filePath}:{log.lineNumber}
      </button>
    </div>
  );
}
```

WebSocketによるリアルタイム配信

```
# バックエンド
from flask_socketio import emit

class EventLogger:
    def log_event(self, event_type, severity, account_id, message, ...):
        event = SystemEvent(...)

        # データベースに保存
        self._store_event(event)

        # WebSocketで配信
        socketio.emit('system_event', {
            'event_type': event.event_type.value,
            'severity': event.severity.value,
            'account_id': account_id,
            'message': message,
            'timestamp': event.timestamp.isoformat()
        })
```

```
// フロントエンド
import { io } from 'socket.io-client';

const socket = io('http://localhost:5000');

socket.on('system_event', (event) => {
  // リアルタイムでログを表示
  addLogToList({
    id: event.id,
    timestamp: event.timestamp,
    accountId: event.account_id,
    level: mapEventSeverityToLogLevel(event.severity),
    message: event.message
  });
});
```

パフォーマンス最適化

インデックス戦略

```
-- アカウント別ログ検索の高速化
CREATE INDEX idx_account_logs_account_id ON account_logs(account_id);

-- 操作種別検索の高速化
CREATE INDEX idx_account_logs_action_type ON account_logs(action_type);

-- ステータス検索の高速化
CREATE INDEX idx_account_logs_status ON account_logs(status);

-- 時系列検索の高速化
CREATE INDEX idx_account_logs_created_at ON account_logs(created_at DESC);

-- リクエストトレーシングの高速化
CREATE INDEX idx_account_logs_request_id ON account_logs(request_id);

-- 複合インデックス(よく使われるクエリ)
CREATE INDEX idx_account_logs_account_status
ON account_logs(account_id, status, created_at DESC);
```

クエリ最適化

最適化前:

```
# 全ログを取得してからフィルタ(非効率)
all_logs = session.query(AccountLog).all()
success_logs = [log for log in all_logs if log.status == 'SUCCESS']
```

最適化後:

```
# データベース側でフィルタ(効率的)
success_logs = session.query(AccountLog).filter(
    AccountLog.status == LogStatus.SUCCESS
).order_by(AccountLog.created_at.desc()).limit(100).all()
```

メモリ管理

```
class EventLogger:
    def __init__(self):
        self.events: List[SystemEvent] = []
        self.max_memory_events = 1000 # メモリに保持する最大イベント数

    def log_event(self, ...):
        event = SystemEvent(...)

        # メモリに保存(最大1000件)
        self.events.append(event)
        if len(self.events) > self.max_memory_events:
            self.events.pop(0) # 古いイベントを削除

        # データベースに保存(永続化)
        self._store_event(event)
```

実装の特徴

1. ゼロコスト抽象化

呼び出し側のコード:

```
# シンプルな使用方法
with log_manager.log_operation(account_id, ActionType.LOGIN) as log_ctx:
    result = perform_login()
    log_ctx.log_success("Login successful")
```

自動的に記録される情報:

- コード位置(file_path, function_name, line_number)
- 実行時間(execution_time)
- リクエストID(request_id)
- タイムスタンプ(created_at)
- エラー詳細(error_detail)

2. 自動エラーハンドリング

```
# エラーが発生しても自動的にログ記録
with log_manager.log_operation(account_id, ActionType.LOGIN) as log_ctx:
    raise Exception("Login failed")
# 自動的にログ記録される(手動でlog_error不要)
```

3. トレーサビリティ

```
# リクエストIDで操作を追跡
request_id = str(uuid.uuid4())

with log_manager.log_operation(account_id, ActionType.LOGIN, request_id=request_id):
    # ログイン処理
    pass

with log_manager.log_operation(account_id, ActionType.SCHEDULE_UPDATE, request_id=request_id):
    # スケジュール更新処理
    pass

# 同じrequest_idで関連する操作を追跡可能
```

セキュリティ対策

1. 機密情報の保護

パスワードの除外:


```
def log_success(self, message, **extra_data):
    # パスワードを自動的に除外
    safe_data = {k: v for k, v in extra_data.items()
                  if k not in ['password', 'password_encrypted']}

    self._store_log(message, safe_data)
```

トークンのマスク:

```
def mask_sensitive_data(data: Dict[str, Any]) -> Dict[str, Any]:
    """機密データをマスク"""
    masked = data.copy()
    if 'session_token' in masked:
        masked['session_token'] = '***'
    if 'api_key' in masked:
        masked['api_key'] = '***'
    return masked
```

2. ログの保護

アクセス制御:

- ログAPIはユーザー認証が必要
- アカウント所有者のみがログを閲覧可能

データ保持期間:

```
# 古いログの自動削除(90日後)
def cleanup_old_logs():
    cutoff_date = datetime.utcnow() - timedelta(days=90)
    session.query(AccountLog).filter(
        AccountLog.created_at < cutoff_date
    ).delete()
```

まとめ

実装した機能

- ✓ 自動コード位置検出: `inspect` モジュールによる自動記録
- ✓ コンテキストマネージャー: 自動エラーログ・自動クリーンアップ
- ✓ イベントシステム: リアルタイムイベント配信
- ✓ 4つの復旧戦略: ログからの状態復元
- ✓ WebSocket統合: リアルタイムログ表示
- ✓ コード位置リンク: フロントエンドから直接コード表示
- ✓ パフォーマンス最適化: インデックス・クエリ最適化

技術スタック

- ログ記録: Python `inspect` モジュール
- データベース: PostgreSQL (Supabase) + JSONB
- リアルタイム配信: Flask-SocketIO + WebSocket
- フロントエンド: React + TypeScript + Socket.IO Client

次のステップ

この基盤の上に、以下の機能が実装されます:

- ログベース復旧の自動化
- 異常検知システム
- ログ分析・レポート機能
- パフォーマンスメトリクスの収集

参考資料

実装ファイル

- **LogManager**: `backend/src/core/log_manager.py`

- **EventSystem**: backend/src/core/event_system.py
- **RecoveryManager**: backend/src/core/recovery_manager.py
- **データベースモデル**: backend/src/database/models.py
- **ログリポジトリ**: backend/src/database/repositories/account_log.py

API統合

- **API Routes**: backend/src/web/api/routes.py
 - GET /api/logs - ログ一覧取得
 - GET /api/logs/<id> - ログ詳細取得
 - GET /api/accounts/<id>/logs - アカウント別ログ取得
 - POST /api/accounts/<id>/recover - ログからの復旧

フロントエンド

- **LogViewer**: frontend/src/components/LogViewer.tsx
- **WebSocket統合**: frontend/src/lib/socket.ts
- **React Query Hook**: frontend/src/hooks/useApi.ts

関連ドキュメント

- **アカウント管理データベース**: docs/milestone2/account_database_detailed_jp.md
- **ブラウザ自動化基盤**: docs/milestone2/browser_automation_detailed_jp.md
- **Milestone 2 完了サマリー**: docs/milestone2/milestone2_summary_jp.md

ドキュメントバージョン: 2.0

最終更新: 2025年12月26日

実装状況: 完全実装完了

技術スタック: Python + TypeScript + Supabase + WebSocket